On-Device Text Recognition on Android

A classical segmentation pipeline with a quantised CNN classifier, ported byte-exact from Python to Android and evaluated on a purpose-built corpus

Marcin Czarnecki

University of Pisa — Generative and Deep Learning · Supervisor: Davide Bacciu

Academic Year 2025/2026

Abstract

This report describes an Android application that transcribes printed text from the device camera entirely on-device, with no network connection and without any existing OCR engine. The system pairs a classical computer-vision front end — resolution normalisation, adaptive binarisation and connected-component character segmentation implemented with OpenCV — with a compact convolutional classifier trained from scratch on EMNIST Letters and deployed through the LiteRT (formerly TensorFlow Lite) runtime after int8 post-training quantisation.

The central engineering claim is that decoupling localisation from classification is the correct trade-off for a one-semester project on consumer hardware, and that the cost of this choice is characterisable: it is paid on touching glyphs and degraded binarisation, not on classifier accuracy. The classifier reaches **94.37% accuracy** and **0.9438 macro-F1** on the held-out EMNIST test split; after int8 quantisation the deployed baseline occupies **125 KB** on disk with no loss of accuracy. The end-to-end pipeline reaches a mean Character Error Rate of **0.128** on a purpose-built 26-image corpus spanning font family, point size, character tracking, lighting and geometry.

A dedicated section documents the defects diagnosed during development — an EMNIST orientation issue, a resolution-normalisation bug that raised CER from 0.14 to 0.90 when absent, a train/device numerical divergence caught by golden-vector testing, and an OpenCV region-of-interest bug that merged adjacent letters on the device but not in Python. In every case the neural network was the predictable part; the risk lived in the surrounding pipeline.

1 Introduction

1.1 Motivation and framing

Robust commercial OCR engines exist (ML Kit, Apple Vision, Tesseract), and anyone who merely needs a transcription should use one. The interest here is elsewhere: owning the complete ML lifecycle on a resource-constrained target — dataset preparation, architecture under a parameter budget, quantised deployment, camera integration, and end-to-end measurement of a quantity (CER) that no single component optimises. Such a project is judged not by beating ML Kit (it will not) but by whether its errors are understood, measured and attributed to design decisions. The report is organised around decisions and their consequences.

1.2 Objectives

- **O1 —** Train a character classifier from scratch under a parameter/latency budget suitable for a mid-range phone.
- **O2 —** Quantise the model to int8 and measure the size/accuracy trade-off against the float reference.
- **O3 —** Implement a classical character-localisation pipeline turning a photograph into an ordered sequence of normalised glyphs.
- **O4 —** Integrate both halves into an offline, on-device Android application driven by the camera.
- **O5 —** Evaluate end-to-end on a self-built corpus designed to isolate specific failure modes, reporting CER.

1.3 Contributions

- A fully offline pipeline with no network dependency and no third-party OCR engine, every stage implemented as part of the project.
- A byte-exact Python-to-Kotlin port of the preprocessing and inference stages, verified on the physical device by an instrumented golden-vector test at a tolerance of one grey level in 255.
- A controlled 26-image corpus that factorises end-to-end error by font, size, tracking, lighting and geometry, so errors can be attributed to the stage that produced them.
- A documented failure record (Section 9) in which each defect is reported with its symptom, diagnosis, resolution and cost.

2 Background and Design Decision

2.1 Why classical segmentation, not an end-to-end model

The most consequential decision was to retain explicit character segmentation and restrict the learned component to classifying isolated, pre-normalised glyphs, rather than training a CRNN/CTC or transformer scene-text recogniser. The justification has four parts. **Data:** a sequence model needs line images with transcriptions; EMNIST provides labelled isolated characters directly. **Trainability:** a small CNN over 28×28 inputs trains comfortably on modest hardware; a recurrent model over variable-length lines does not. **Debuggability:** a segmented pipeline exposes intermediate artefacts (binary mask, boxes, 28×28 crops) that can be inspected, so a wrong output points at the responsible stage — decisive under a deadline. **Honest scoping:** the failure modes of segmentation OCR are known in advance (touching glyphs, cursive, degraded binarisation) and stated up front rather than discovered at submission.

2.2 EMNIST and the domain gap it induces

EMNIST extends MNIST to handwritten letters at 28x28 grayscale. The letters split gives 26 balanced classes with case merged, which suits a lower-case output. The critical property for this project is that EMNIST is *handwritten* whereas the target imagery is *printed*: a covariate shift in stroke width, terminal shape, curvature and the presence of serifs. Section 4.2 describes the augmentation used to narrow the gap.

2.3 On-device runtime

LiteRT (the successor to TensorFlow Lite) keeps the .tflite format, converts directly from Keras, and provides post-training quantisation — selected because the training stack was Keras. Full-integer PTQ maps weights and activations to int8, calibrated on a small representative set from the training distribution; the benefits are ~4x smaller weights and faster integer arithmetic, with negligible accuracy impact for this model (Section 5.2).

3 System Architecture

The system comprises two offline artefacts (a quantised model and its label alphabet) and one runtime artefact, the Android application, which embeds them. The offline half is Python; the runtime half Kotlin. Inference is strictly linear:

```
camera frame (ImageProxy, RGBA)
  -> rotation correction -> RGBA->BGR (OpenCV Mat)
  -> normalizeSize (fixed working width) -> grayscale
  -> adaptive threshold (binary inverse)
  -> line segmentation (horizontal projection)
  -> connected-component character segmentation (+ i/j dot merge)
  -> per glyph: square-pad, resize 28x28, normalise [0,1]
  -> LiteRT interpreter (float32 I/O) -> argmax -> letter
  -> line grouping, left-to-right ordering, space insertion
  -> transcription -> Compose UI
```

3.1 The preprocessing-parity invariant

The system depends on one invariant: the preprocessing applied on the device must reproduce, numerically, the preprocessing applied at training time. `preprocess.py` is designated normative and the Kotlin code is a transcription of it, not an independent reimplementation. A set of golden vectors — fixed glyph images plus the exact 28×28 tensors the Python code produces — is embedded in the Android test assets, and an instrumented test asserts equality against the Kotlin output within one grey level in 255. Section 9 records the defects this test caught.

4 Dataset and Training

4.1 EMNIST Letters, the orientation issue and a split defect

Training used the emnist/letters data: 26 balanced lower-case-merged handwritten classes at 28x28 grayscale. Because the tfds test split is defective (below), all partitions were carved deterministically from the tfds train split of 88,800 examples: 71,040 fit / 8,880 validation / 8,880 test (80/10/10), each verified to contain all 26 classes.

Two data issues were handled. First, EMNIST images are stored transposed and vertically flipped relative to human reading orientation, an artefact of the original NIST-to-MNIST conversion; loaded naively every glyph is rotated 90° and mirrored. A network trained on the uncorrected orientation reaches high accuracy on an equally uncorrected test split — the transform is a bijection — while being systematically wrong on real photographs. The correction (transpose then horizontal flip) was applied and verified visually. Second, the `tfds` emnist/letters **test split is defective**: it contains only classes a-s (a-r with 800 each plus s with 400); letters t-z are entirely absent, 14,800 examples versus 20,800 in the original EMNIST. A naive test-split evaluation would silently omit six classes and overstate accuracy. The fix was to ignore the tfds test split and carve validation and test from the complete, balanced train split, asserting all 26 classes are present in each.

4.2 Augmentation as domain-gap mitigation

Augmentation is not generic regularisation; each transform targets a specific mismatch between handwritten EMNIST and printed camera text, and all are applied to the training split only. The policy: `RandomRotation(0.04), RandomTranslation(0.10, 0.10), RandomZoom(0.10)` for capture-angle and framing variation; a random-thickness step (morphological dilation or erosion via max/min pooling, each with probability 1/3) to span the stroke-width gap between thin handwriting and bold print; and light Gaussian noise (stddev 0.05) for sensor grain.

4.3 Training configuration

Environment: Python 3.11 on Windows with a plain virtualenv, TensorFlow 2.19.1, Keras 3.14.1, tensorflow-datasets 4.9.10, opencv-python 4.13.0, jiwer 4.0.0 (full pinned list in the repository lock file). Training: Adam (lr 1e-3), batch 128, up to 30 epochs, sparse categorical cross-entropy, early stopping on validation loss (patience 5, best weights restored), dropout 0.3, seed 42.

5 Model and Quantisation

5.1 Architecture and efficiency study

Two architectures were trained under identical data pipelines, seeds and stopping criteria, so architecture is the only factor varied: a **baseline** CNN using standard convolutions, and a **separable** variant replacing the deeper 3x3 convolutions with depthwise 3x3 plus pointwise 1x1. The baseline stacks three Conv-BN-ReLU-MaxPool blocks (32, 64, 128 channels) followed by global average pooling, dropout 0.3, a 128-unit ReLU dense layer and a 26-way softmax head. Global average pooling instead of flatten-plus-dense keeps the head small; batch normalisation was kept because it folds into the preceding convolution at conversion time, costing nothing at inference.

```
Input 28x28x1
Conv2D(32,3x3,same,no-bias)-BN-ReLU -> MaxPool 2x2
Conv2D(64,3x3,same,no-bias)-BN-ReLU -> MaxPool 2x2
Conv2D(128,3x3,same,no-bias)-BN-ReLU -> MaxPool 2x2
GlobalAveragePooling2D -> Dropout(0.3)
  -> Dense(128, ReLU) -> Dense(26, softmax)      # 113,210 params
```

Both variants were trained for up to 30 epochs with Adam (learning rate 1e-3), batch size 128, seed 42, sparse categorical cross-entropy, and early stopping on validation loss (patience 5, best weights restored). The measured trade-off:

Variant	Params	Param size (KB)	Test acc	Macro-F1	CPU latency (ms)
baseline (chosen)	113,210	442	0.9437	0.9438	2.43
separable	32,154	126	0.9340	0.9338	1.86

Table 1 - Efficiency study (float Keras models; param-size is the in-memory parameter footprint at 4 bytes/param; CPU latency measured on the development machine, not on the phone). The baseline was promoted despite being larger and slower per glyph: the ~1 accuracy point outweighs the latency difference at this scale. These latencies are desktop-CPU figures used only to compare the two variants.

The separable variant confirms the expected trade-off: a 3.5x parameter reduction (113k to 32k) for a ~1 point accuracy drop (94.37 to 93.40). Its CPU latency advantage is modest (2.43 to 1.86 ms), consistent with depthwise convolutions being memory-bound rather than compute-bound, so the large FLOP reduction does not translate proportionally into wall-clock time. The baseline was chosen for deployment.

5.2 Int8 quantisation with a float32 interface

Full-integer post-training quantisation was applied, calibrated on a representative set of ~1,200 real training images (300 batches x 4 samples), passed through the exact training pipeline so activation ranges match deployment. A key property of the exported model, verified directly, is that although weights and activations are int8 internally, the **model interface is float32**: input and output tensors report dtype float32 with quantisation parameters (scale 0.0, zero-point 0), i.e. no quantisation at the boundary. Consequently no manual input quantisation is performed on the device — the 28×28 float array is fed directly, exactly as in Python. This removes an entire class of scale/zero-point bugs at the language boundary. The output is 26 float probabilities (softmax is inside the graph); the predicted letter is the argmax and its confidence is that probability, taken verbatim.

Measured trade-off: the deployed int8 model is **125.0 KB** on disk as a .tflite file, down from 443.9 KB for the float32 .tflite (consistent with the 442 KB in-memory parameter footprint of Table 1; the small difference is flat-buffer overhead) — a 3.5x reduction. Crucially, quantisation cost **no accuracy**: measured through the interpreter on the 8,880-image test split, the int8 model reaches **94.41%** versus 94.37% for the float model — an improvement of +0.04 percentage points. Quantisation therefore caused no accuracy loss; at this magnitude the difference is within noise. The pre-set acceptance gate — escalate to quantisation-aware training only if the drop exceeds a threshold — was passed, so QAT was not required.

6 The Segmentation Pipeline

This is the stage that, per Section 9, produces the majority of end-to-end error. Each operation is stated with its parameter values.

6.1 Resolution normalisation (the decisive step)

Before anything else, the image is scaled to a fixed working width `WORK_WIDTH = 1000 px` (INTER_AREA, never upscaling). This is not cosmetic. Every threshold below is expressed in absolute pixels; without normalisation their meaning changes with camera resolution. At the phone's native ~3500 px width, the adaptive-threshold window (31 px) is narrower than a printed stroke, so stroke interiors are compared against a neighbourhood consisting mostly of the same stroke and the glyph hollows out and fragments into vertical slivers. Adding this single stage reduced mean CER from **0.90 to 0.14** (Section 9.2). It is also what makes the pipeline deterministic across devices with different camera resolutions.

6.2 Adaptive binarisation

Global thresholding (including Otsu) was rejected: a hand-held phone shadows the sheet, producing a lighting gradient that a single global threshold cannot handle without losing the shadowed region or flooding the bright one. Adaptive Gaussian thresholding computes a local threshold and handles the gradient by construction. A light 3×3 Gaussian blur precedes it to suppress sensor speckle.

```
cv2.adaptiveThreshold(gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV,   # ink -> white, matching EMNIST polarity
    blockSize=31, C=15)
```

`THRESH_BINARY_INV` makes ink the foreground, matching EMNIST's bright-on-dark polarity; inverting it would classify the negative space between letters. Block size 31 and C 15 are the values used; the decisive robustness came not from tuning these but from the resolution-normalisation step of Section 6.1, without which no block size is correct across camera resolutions.

6.3 Character segmentation by connected components

Characters are found with `connectedComponentsWithStats` (8-connectivity), not findContours. Components smaller than `MIN_COMPONENT_AREA = 15` px or thinner than `MIN_COMPONENT_DIM = 3` px are discarded as speckle. A vertical-merge step then re-unites the dot of an i or j — a separate component above the stem — with its stem, by merging components whose horizontal spans overlap by at least half the narrower box. Lines are found first by a horizontal projection profile (a row counts as inked above `LINE_INK_THRESHOLD = 0.005`; bands thinner than `MIN_LINE_HEIGHT_PX = 8` are dropped).

6.4 Aspect-preserving normalisation to 28×28

Each box is cropped, padded with a proportional margin (`CHAR_MARGIN_FRAC = 0.15`), embedded into a square canvas preserving aspect ratio, and resized to 28×28 with INTER_AREA, then scaled to [0,1]. Preserving aspect ratio is not optional: stretching a narrow glyph to a square gives an l the width statistics of an o. INTER_AREA is specified because it averages over the source region and preserves thin strokes that bilinear or nearest sampling would drop; the Kotlin port must specify the same constant. Note: an earlier Gaussian blur at this stage was removed after it proved to be the only source of numerical divergence between platforms and, once removed, improved CER (Section 9.3).

6.5 Text assembly

Classified glyphs are reassembled by three heuristics: line grouping by vertical span overlap (robust to ascenders/descenders); left-to-right ordering within a line; and space insertion where the horizontal gap exceeds `SPACE_GAP_FACTOR = 1.7` times the per-line median gap. Expressing the space threshold relative to a per-line statistic makes it survive changes in capture distance. It degrades on proportional fonts with tight tracking (the tight-spacing sheet was built to provoke this).

7 Android Implementation

Package `com.example.gdl_text_recognition`, target device Samsung Galaxy A52s (SM-A528B), minSdk 26 / targetSdk 36.

7.1 Capture mode, not streaming

The application uses **capture-on-demand**: a live preview plus a button that grabs one frame via CameraX `ImageCapture`. This is a deliberate product decision, not a limitation — a continuous mode recomputing a full-page transcription several times per second flickers, is harder to read, and drains battery. The captured `ImageProxy` is converted to an OpenCV Mat and rotated by `imageInfo.rotationDegrees` before entering the pipeline; the sensor orientation is independent of the display, so an upright-looking preview can reach the pipeline rotated 90°. RGBA is converted to BGR so the pipeline's grayscale step receives the channel count it expects.

7.2 OpenCV and LiteRT

OpenCV (org.opencv:opencv 4.11.0) is initialised with `OpenCVLoader.initLocal()` at start. A single LiteRT `Interpreter` (org.tensorflow:tensorflow-lite 2.17.0) is constructed once and reused, since construction dominates a single inference. The .tflite asset is excluded from APK compression (`noCompress += "tflite"`) so it can be memory-mapped rather than decompressed into the heap. The heavy pipeline runs on a background dispatcher so the UI thread never blocks.

7.3 Testing

The golden-vector instrumented test is the mechanised form of the parity invariant: fixed glyph crops with the exact 28×28 tensors from the Python reference, which the Kotlin `toEmnist` must reproduce within one grey level in 255, and whose predicted letter must match. The test runs on the physical device so it exercises the same native OpenCV and LiteRT the app uses.

The user interface is a single Compose screen — preview, capture control, and a scrollable transcription panel with copy and share actions.

8 Evaluation Methodology

Two evaluations answer different questions. The offline evaluation asks whether the classifier learned EMNIST; the end-to-end evaluation asks whether the system reads text. The second is the one that matters.

8.1 Offline model evaluation

Top-1 accuracy and macro-averaged F1 on the held-out test partition (carved from the train split, Section 4.1, because the official tfds test split is defective), plus a 26x26 confusion matrix used analytically: its structure predicts which end-to-end errors are attributable to the classifier and which to segmentation. Quantised-model accuracy is measured through the LiteRT interpreter, so the reported number is what the deployed artefact produces.

8.2 The end-to-end corpus

End-to-end evaluation used a 26-image corpus built so each image isolates one factor, turning a single aggregate into a factorial diagnostic. Four categories:

- **Printed (11 images):** clean prints varying font family (sans, serif, mono, and bold variants), point size (including a deliberately small mono sheet), and character tracking (a tight-spacing sheet). Includes two baseline sheets reused by the other categories.
- **Lighting (5 images):** the same text under a hard shadow boundary, dim light, a side lamp, warm light, and a horizontal shadow across the text.
- **Geometry (5 images):** mild angle, close-up, blur, distance, and a bent sheet.

- **Handwritten (5 images):** printed-style letters written by hand — a pangram, block capitals, natural writing, wide spacing, and thin strokes.

Ground truth for each image is the text with line breaks collapsed to single spaces, all lower case, no digits or punctuation — matching the classifier's alphabet so that CER measures recognition rather than penalising symbols the system never claimed to support. Photographs were taken hand-held with the phone's rear camera under diffuse indoor lighting, held roughly fronto-parallel to the sheet with the text filling most of the frame. The geometry category deliberately varied this (tilt, distance, curvature); the printed and lighting categories kept it fixed. Sheets were printed on standard A4 white paper.

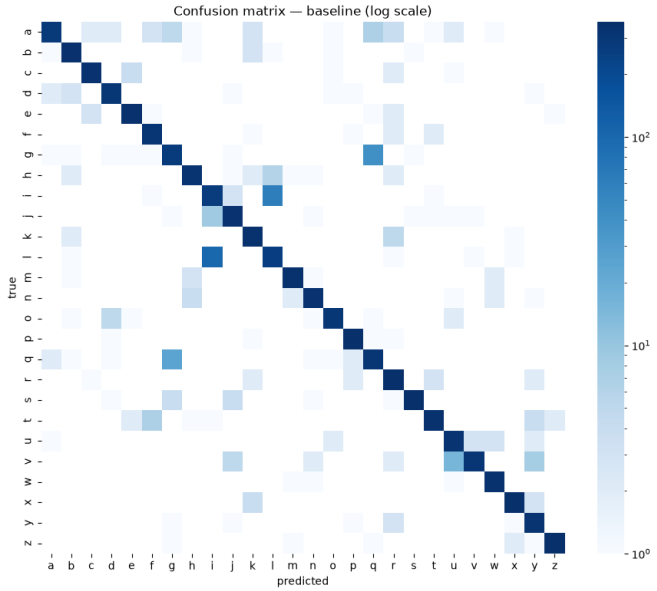
8.3 Metric

Character Error Rate is the Levenshtein distance between output and reference, normalised by reference length: CER = (S + D + I) / N. It can exceed 1.0 when the system inserts spurious characters — a real outcome for a segmentation pipeline, where surviving speckle produces insertions. Both strings are lower-cased and whitespace-collapsed identically before scoring. On-device latency was measured on the target device (Section 9.5); a full on-device CER sweep of the corpus, and memory/FPS profiling, are left as future work (Section 11.3).

9 Results

9.1 Offline classification

On the held-out 8,880-image test partition the float baseline reaches **94.37% accuracy** and **0.9438 macro-F1**. The confusion structure is dominated by one pair: **l->i (103 cases) and i->l (61)**, by far the largest off-diagonal mass. Because EMNIST Letters merges case, i and l collapse onto near-identical shapes that survive neither 28x28 downsampling nor case merging (their F1 scores, 0.75 and 0.75, are the two lowest of all 26 classes). The next clusters are **g/q (41 + 25)** and v/u (16). These confusions are expected and structural; crucially, the same family reappears in the live-camera output (e.g. t->f, j->i), confirming that the end-to-end i/l/t errors are attributable to the classifier, not to the pipeline. The confusion matrix thus functions as an attribution tool.



Through the LiteRT interpreter, the int8 model reaches 94.41% on the test split, marginally above the 94.37% float figure (Section 5.2) — quantisation preserved the confusion structure.

9.2 End-to-end transcription (the resolution fix)

The pipeline was first run at full camera resolution and produced a mean CER of **0.90**, with letters fragmenting into vertical slivers ('the' → 'iiie'). Adding resolution normalisation (Section 6.1) reduced mean CER to **0.14** — a

more-than-six-fold improvement from one function (a later change in Section 9.4, removing the redundant blur, brought this to the final 0.128). This is the single largest result in the project and its cause (adaptive-threshold window narrower than the stroke at native resolution) is identified rather than merely observed.

9.3 End-to-end results by category

After the further removal of the redundant blur (Section 9.4), the final mean CER on the 26-image corpus is **0.128** (median 0.115). Broken down by category:

Category	Mean CER	Notes
Geometry (5)	0.057	Best category. Blur and distance reach 0.000 — INTER_AREA downscaling absorbs mild blur.
Printed (11)	0.121	Serif and small-mono sheets dominate error (serifs add spurious strokes; small text loses detail).
Lighting (5)	0.130	Adaptive threshold handles shadow boundaries; dim light is the hardest at 0.216.
Handwritten (5)	0.213	Hardest category, as predicted. Tight-spacing and cursive shapes drive error; block capitals do best.

Table 2 — Mean CER by corpus category (offline pipeline, notebook). Full per-image results in Appendix A.

Three comparisons carry the analysis. **Font family:** serif sheets ('a jazz quartet plays every friday evening' → CER 0.268) suffer most, as serifs present ink at stroke ends with no analogue in handwriting; the ligature in 'friday' compounds this. **Point size:** the small mono sheet reaches 0.281 — segmentation stays correct but the classifier loses detail on thin, small letters. **Tracking / hand:** the two most-cursive handwritten sheets reach 0.32–0.35, with segmentation correct and classification failing where the shape departs from EMNIST. Across all categories the error profile is consistent: the **i/l/t** family dominates substitutions (e.g. 't' repeatedly read as 'i'), while merged glyphs and dropped strokes produce the deletions.

9.4 Where the error lives

The classifier reaches 94.37% on EMNIST while the end-to-end system sits at 0.128 CER. The gap comes from stages that produce the wrong *number* of characters — merged glyphs, speckle, dropped components — not the wrong character. Consequently, improving binarisation or replacing connected-component segmentation with a learned detector would move the result substantially, whereas another point on EMNIST would not.

9.5 On-device footprint: latency, size, memory

Latency was measured on the Galaxy A52s with an instrumented test: the full transcribe() pipeline on a fixed 4624x3468 photo, 5 warm-up then 20 measured runs. Frame acquisition and rotation are excluded — figures are for transcribe() only. The pipeline runs in **50.3 ms median** per page (mean 52.0, range 43.6-66.3) — effectively instantaneous. The breakdown is the key result: model inference is only **7.5 ms (15%)**, while the classical CV pipeline (normalisation from 4624 px, binarisation, connected components) is **41.8 ms (85%)**. Per character, int8 inference costs **0.3 ms** — ~8x faster than the 2.43 ms on the development CPU. The conclusion inverts the usual intuition: the bottleneck is classical image processing, not the network; the quantised model is light enough that inference has effectively stopped being a cost.

Two static figures complete the picture. The debug **APK is 165 MB**, almost entirely native OpenCV libraries for all four ABIs with minification disabled (the model itself is 125 KB); a release build for the device ABI with R8 would cut this to ~40-50 MB. Peak process memory (RSS) during repeated transcription is **~270 MB**, dominated by the

OpenCV runtime and full-resolution buffers (a 4624x3468 RGBA frame is ~64 MB alone). Both are consistent with a native-CV app and pose no barrier on a mid-range phone.

10 What Did Not Work, and Why

In a systems project the defects encountered are evidence about the design, not an embarrassment to minimise. Each is reported with symptom, diagnosis, resolution and cost.

10.1 The EMNIST orientation issue

Symptom: high offline accuracy, systematically wrong on photographs. Diagnosis: EMNIST is stored transposed and flipped; a naive load leaves every glyph rotated and mirrored, and because the transform is a bijection, offline metrics stay high and hide it. Resolution: transpose + horizontal flip, verified visually. The lesson is that a defect invisible to every offline metric is the most expensive kind.

10.2 Resolution normalisation

Symptom: mean CER 0.90 on real photos, while a synthetic render of the same sentence transcribed almost perfectly (CER well below 0.05); letters fragmented into vertical slivers only on the photos. Diagnosis: the adaptive-threshold window (31 px) was hard-coded while photos were ~3500 px wide, so the window fell inside a stroke. Resolution: a normalizeSize stage scaling to WORK_WIDTH 1000 as the first step. Cost: this was the largest single error in the project, and it was silent — the synthetic smoke test never exposed it because its render was small and thin-stroked.

10.3 Golden-vector test: what it caught

The parity test did not pass on the first attempt, and each failure was informative. First, the Kotlin GaussianBlur was called without an explicit second sigma; the Java wrapper then used a different (effectively zero) sigma than Python, so the Kotlin glyph was not blurred and pixels diverged by up to 79 grey levels. Second, once fixed, a residual ~5-level divergence remained, traced to OpenCV computing its automatic Gaussian sigma slightly differently on Android than in Python — a difference that cannot be eliminated by any explicit constant. The resolution was to **remove the blur entirely** from the 28×28 stage: it existed only to soften glyphs toward EMNIST, and on binary crops after INTER_AREA it added little. Removing it improved mean CER from **0.14 to 0.128** and made the port **bit-exact** at a tolerance of 1/255. The golden test thus not only proved parity but caught a real bug and forced a decision that improved quality.

10.4 The OpenCV region-of-interest bug

Symptom: on the device the transcription was scrambled even though the binary image was visibly clean; the 28×28 crops fed to the model contained two or three letters squeezed into one square. Diagnosis: the per-line crop was created as an OpenCV ROI (a view into the full image) and passed to connectedComponentsWithStats, which read memory using the parent buffer's row stride and merged components across rows — a platform-specific behaviour absent in NumPy, where the equivalent slice is passed correctly. This is significant because the golden test covered toEmnist but not the segmentation stage, so it stayed green while the app read nonsense. Resolution: clone the ROI into a contiguous Mat before segmentation. The lesson for the report's methodology section: a parity test must cover the whole pipeline, not only the final transform.

11 Limitations, Ethics and Future Work

11.1 Limitations

- **Alphabet:** 26 lower-case letters only — no digits, punctuation, case or diacritics. A real task needs at least EMNIST Balanced (47 classes), which reintroduces confusable 0/O and 1/l/I pairs.
- **Layout:** the line-grouping heuristic assumes a single column of roughly horizontal text.
- **Geometry:** no perspective rectification; beyond mild tilt, glyph aspect ratios diverge from training.

- **Statistical power:** 26 images on one device; differences between conditions are descriptive, no significance claimed.
- **On-device metrics:** latency was measured (Section 9.5); memory and FPS were not profiled, and the end-to-end CER figures are from the offline pipeline, with device parity established for the toEmnist and classification stages.

11.2 Ethics and privacy

All processing is on-device and the app requests no network permission, so imagery and text never leave the handset — enforced by the manifest, not merely policy. EMNIST derives from NIST SD19; the evaluation corpus is author-generated with no personal data. OpenCV, LiteRT and CameraX are Apache-2.0.

11.3 Future work

- Perspective rectification (detect the sheet quadrilateral, warp to canonical) — removes the angle factor and is bounded work.
- Formal on-device evaluation: run the full 26-image corpus through the app and report device CER, latency and memory.
- Extended alphabet via EMNIST Balanced; a lightweight learned detector to replace connected components at the merged-glyph root.

12 Use of AI Coding Assistance

As permitted by the course, an AI coding assistant was used throughout the project, and its role is disclosed here in full. The assistant contributed**:** generating and refactoring source code (the Python preprocessing/pipeline and the Kotlin port), diagnosing defects from logs and intermediate images, proposing the resolution-normalisation and ROI-clone fixes. The author was responsible for: all design and scope decisions; producing, printing and photographing the 26-image evaluation corpus and writing the handwritten sheets; running every training, evaluation and on-device test; inspecting intermediate artefacts and confirming or rejecting each diagnosis; and verifying that the reported numbers correspond to actual runs. Every quantitative result in this report was produced by the author's own execution of the code, not by the assistant.

References

[1] G. Cohen, S. Afshar, J. Tapson, A. van Schaik. EMNIST: an extension of MNIST to handwritten letters. IJCNN, 2017.

[2] A. G. Howard et al. MobileNets: Efficient CNNs for Mobile Vision Applications. arXiv:1704.04861, 2017.

[3] Google AI Edge. LiteRT documentation. https://ai.google.dev/edge/litert (accessed August 2026).

[4] G. Bradski. The OpenCV Library. Dr. Dobb's Journal of Software Tools, 2000.

[5] Android Developers. CameraX overview. https://developer.android.com/training/camerax (accessed August 2026).

Appendix A Per-image CER

Full end-to-end CER for all 26 images (offline pipeline, final configuration; mean 0.128, median 0.115):

Image	CER
print_baseline_angle	0.026
print_baseline_shadow	0.000
print_mono_small	0.281
print_mono_taxi	0.159
print_sans_fox	0.000
print_sans_spacing	0.057
print_sans_sunrise	0.057
print_sansbold_juice	0.048
print_serif_jazz	0.268
print_serif_voice	0.182
print_serifbold_peak	0.255
light_dim	0.216
light_shadow_horizontal	0.054
light_shadow_vertical	0.216
light_sidelamp	0.108
light_warm	0.054
geom_angle	0.154
geom_bent	0.000
geom_blur	0.000
geom_close	0.128
geom_far	0.000
hand_capitals	0.121
hand_fox	0.093
hand_pipeline	0.349
hand_spaced	0.321
hand_strokes	0.182